

Tipi di dato

Marco Cerbella

Lezione 8 – 7 ottobre 2026

1 Tipi e oggetti

I programmi devono memorizzare ed elaborare dati di tipo diverso, come interi e numeri in virgola mobile, in modi diversi: il compilatore deve quindi sapere che tipo di dato rappresenta un valore.

Definizione. In C un *oggetto* è una zona di memoria il cui contenuto può rappresentare valori. Gli oggetti che hanno un nome sono detti *variabili*. Il *tipo* di un oggetto determina:

- quanto spazio occupa in memoria;
- i valori che la variabile può assumere;
- le operazioni che si possono eseguire su di essa.

1.1 Tipi in C

- **Tipi di base:** interi (standard ed estesi) e floating-point (reali e complessi).
- **Tipi enumerati.**
- Il tipo `void`.
- **Tipi derivati:** puntatori, array, strutture, unioni, funzioni.

I tipi di base e i tipi enumerati formano i *tipi aritmetici*; i tipi aritmetici e i puntatori formano i *tipi scalari*; array e strutture sono detti *tipi aggregati*. Un *tipo funzione* descrive l'interfaccia di una funzione: il tipo del valore restituito ed eventualmente i tipi dei parametri.

2 Interi

I tipi interi con segno sono cinque (molti hanno sinonimi):

tipo	sinonimi
<code>signed char</code>	
<code>int</code>	<code>signed</code> , <code>signed int</code>
<code>short</code>	<code>short int</code> , <code>signed short</code> , <code>signed short int</code>
<code>long</code>	<code>long int</code> , <code>signed long</code> , <code>signed long int</code>
<code>long long</code> (C99)	<code>long long int</code> , <code>signed long long</code> , <code>signed long long int</code>

C definisce solo le dimensioni minime: `short` occupa almeno 2 byte, `long` almeno 4 e `long long` almeno 8. I tipi possono essere più grandi, ma devono rispettare l'ordine

$$\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long}).$$

2.1 Booleani

In C non esistono *true* e *false*: 0 è falso e qualsiasi valore diverso da 0 (ad esempio 1, -25, 123456) è vero.

```
1  if (3)                if (0)
2      printf("YES\n");    printf("YES\n");
3  else                  else
4      printf("NO\n");      printf("NO\n");
```

Il primo `if` stampa YES, il secondo NO.

C99 ha introdotto il tipo intero senza segno `_Bool`: *vero* è codificato come 1, *falso* come 0. Includendo l'header `stdbool.h` si possono usare anche `bool`, `true` e `false`: `bool` è sinonimo di `_Bool` e `true`, `false` sono costanti uguali a 1 e 0.

2.2 Caratteri

Anche `char` è un tipo intero standard, ma il nome `char` è sinonimo di `signed char` oppure di `unsigned char` a seconda del compilatore. Occupa 1 byte e i caratteri corrispondono ai codici della tabella ASCII.

Con le variabili `char` si può fare aritmetica: sta al programmatore decidere se il numero contenuto è un codice di carattere o altro.

```
1  char ch = 'A';        // variabile di tipo char
2  printf("The character %c has the character code %d.\n", ch, ch);
3  printf("%c", ch + 1);
```

```
The character A has the character code 65.
B
```

3 Sistemi di numerazione

Definizione. Il sistema *binario* (base 2) rappresenta i numeri con due soli simboli, 0 e 1; ogni cifra è un *bit*. Quasi tutti i calcolatori lo usano internamente perché è semplice da realizzare con circuiti a porte logiche.

Definizione. Il sistema *esadecimale* (base 16) usa sedici simboli: le cifre 0–9 per i valori da zero a nove e le lettere A, B, C, D, E, F (o minuscole) per i valori da dieci a quindici. Ogni cifra esadecimale corrisponde a 4 bit (un *nibble*, metà di un byte da 8 bit): è una rappresentazione più leggibile dei valori binari. Il sistema *ottale* (base 8) usa le cifre 0–7.

Bin.	Esa.	Dec.	Bin.	Esa.	Dec.
0	0	0	1000	8	8
1	1	1	1001	9	9
10	2	2	1010	A	10
11	3	3	1011	B	11
100	4	4	1100	C	12
101	5	5	1101	D	13
110	6	6	1110	E	14
111	7	7	1111	F	15

3.1 Da binario/esadecimale a decimale

Un numero con cifre $d_n \dots d_1 d_0$ in base b vale $d_n b^n + d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0$ (in base 2: $d_n 2^n + \dots + d_1 2^1 + d_0 2^0 = N_{10}$).

Esempio. $1001_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 9$ e $002B_{16} = 2 \cdot 16 + 11 = 43$.

3.2 Da decimale a binario/esadecimale

Si divide ripetutamente il numero per la base e si leggono i resti *dal basso verso l'alto*. Per l'ottale si usa lo stesso algoritmo con base 8.

Esempio. 156 in binario (divisioni per 2): i quozienti successivi sono 78, 39, 19, 9, 4, 2, 1, 0 e i resti 0, 0, 1, 1, 1, 0, 0, 1. Letti dal basso: $156 = 10011100_2$.

1565 in esadecimale (divisioni per 16): $1565 = 97 \cdot 16 + 13$ (resto 13 = d), $97 = 6 \cdot 16 + 1$ (resto 1), $6 = 0 \cdot 16 + 6$ (resto 6). Quindi $1565 = 61d_{16}$.

Con N bit si rappresentano i numeri da 0 a $2^N - 1$; con 8 bit (un byte) da 0 a 255 (da 0000 0000 a 1111 1111). Servono però anche i numeri negativi, e per questo esistono rappresentazioni diverse: segno e modulo, complemento a due.

4 Rappresentazione degli interi in memoria

4.1 Segno e modulo

Un bit (di solito il più a sinistra) indica il segno: 0 per i positivi, 1 per i negativi; i bit restanti rappresentano il modulo. Ad esempio 1001 1000 vale -24 (mentre se si usasse solo per numeri positivi varrebbe 152).

Con n bit si rappresentano i valori da $-(2^{n-1} - 1)$ a $2^{n-1} - 1$, e ± 0 : con 8 bit da -127 a $+127$.

Osservazione. Problema: lo zero ha due rappresentazioni, 0000 0000 (+0) e 1000 0000 (−0). Una soluzione è il complemento a due.

4.2 Complemento a due

Valore binario	Complemento a due	Senza segno
00000000	0	0
00000001	1	1
$\vdots$	$\vdots$	$\vdots$
01111110	126	126
01111111	127	127
10000000	-128	128
10000001	-127	129
10000010	-126	130
$\vdots$	$\vdots$	$\vdots$
11111110	-2	254
11111111	-1	255

Nel complemento a due esiste un solo zero, 00000000. Per cambiare segno a un numero (positivo o negativo) si **invertono tutti i bit e poi si somma 1**.

Esempio.

- 0000 0101 (valore 5); invertendo: 1111 1010; sommando 1: 1111 1011, che vale -5.
- Da 1111 1011: invertendo 0000 0100, sommando 1 0000 0101, cioè di nuovo 5.

Con n bit si rappresentano i valori da -2^{n-1} a $2^{n-1} - 1$: non c'è "-0", quindi si rappresenta un numero negativo in più. Con 8 bit, da -128 (1000 0000) a +127 (0111 1111).

Osservazione. La regola "inverti e somma 1" non funziona per -128, perché 128 non è rappresentabile con 8 bit in complemento a due. D'ora in poi si usa il complemento a due.

4.3 Endianness

Definizione. L'*endianness* indica l'ordine usato per interpretare numericamente una sequenza di byte della memoria come una parola più grande (e l'ordine di trasmissione dei byte su un collegamento digitale). Le parole possono essere in formato *big-endian* o *little-endian*, a seconda che si parta dall'estremità più significativa (big end) o meno significativa (little end).

Esempi: i mainframe IBM z/Architecture e i Motorola 68000 usano big-endian, i processori Intel x86 little-endian.

La memoria è un array di byte; ogni byte ha il suo *indirizzo logico* (un numero positivo), cioè l'indirizzo al quale un elemento appare risiedere dal punto di vista di un programma in esecuzione. Per architetture a 64 bit il limite superiore è $2^{64} - 1$.

Esempio. Il numero 2 scritto su 4 byte, a partire dall'indirizzo 1500:

	1500	1501	1502	1503
big-endian	00	00	00	02
little-endian	02	00	00	00

Esempio. Sulla stringa di 4 bit 0010, in complemento a due: in big-endian vale 2; in little-endian si leggono i bit in ordine inverso (0100) e vale 4. Sulla stringa 1010: in big-endian vale -6, in little-endian (0101) vale 5.

5 Dimensione e limiti dei tipi

L'operatore `sizeof` restituisce la dimensione esatta in byte di un tipo o di una variabile: `sizeof (tipo)` oppure `sizeof espressione` (in questo caso conta il tipo dell'espressione).

```
1 int iIndex;
2 iIndex = 1000;
3 // sizeof(int) e sizeof(iIndex) valgono entrambi 4
```

I limiti dei tipi interi per il proprio compilatore sono nell'header `limits.h`, che definisce macro come `INT_MIN`, `INT_MAX`, `UINT_MAX`, `CHAR_MIN`, `CHAR_MAX`.

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 int main() {
5     printf(" char %zu %d %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
6     printf(" int %zu %d %d\n", sizeof(int), INT_MIN, INT_MAX);
7     return 0;
8 }
```

```
char 1 -128 127
int 4 -2147483648 2147483647
```

Osservazione. Il risultato di `sizeof` ha tipo `size_t`: per stamparlo è corretto `%zu` (nelle slide compare `%d`). I valori stampati dipendono dalla piattaforma (qui un caso tipico).

6 Numeri in virgola mobile

Per rappresentare i non interi, con la virgola in qualsiasi posizione, C ha tre tipi floating-point standard:

- `float`: precisione singola;
- `double`: precisione doppia;
- `long double`: precisione estesa.

6.1 Precisione

Un valore in virgola mobile si memorizza solo con una *precisione* limitata, determinata dal formato binario e dalla memoria usata. La precisione si esprime in cifre significative: ad esempio con sei cifre significative la conversione in un numero decimale di sei cifre restituisce le sei cifre originali. La posizione della virgola e gli zeri iniziali o finali non contano: 123456000 e 0,00123456 si possono entrambi memorizzare in un tipo con precisione a sei cifre.

In C le operazioni aritmetiche tra numeri in virgola mobile sono eseguite internamente con precisione double o superiore.

```
1 float height = 1.2345, width = 2.3456;    // precisione singola
2 double area = height * width;              // il calcolo è fatto in double
```

Se il risultato è assegnato a una variabile `float`, viene arrotondato se necessario. L'header `float.h` definisce macro come `FLT_MIN`, `FLT_MAX` e `FLT_DIG`, che indicano intervallo di valori e precisione del tipo `float`.

6.2 Notazione E e formato IEEE 754

La *notazione E* è la rappresentazione testuale della notazione scientifica: `1.234e+56` significa $1,234 \times 10^{56}$.

Nel formato *IEEE 754* ogni numero finito è descritto da tre interi: un segno s (zero o uno), un *significando* (o mantissa) c e un esponente q . Il valore è

$$(-1)^s \times c \times b^q,$$

dove b è la base (2 o 10). Ad esempio con base 10, segno 1 (negativo), significando 12345 ed esponente -3 il valore è $-12,345$.

Il formato 754 a *precisione singola* usa: 1 bit di segno, 8 bit di esponente, 23 bit di significando. Il valore è

$$(-1)^{\text{segno}} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right) \times 2^{(e-127)}.$$

Esempio. Con segno 0, esponente 01111100 ($e = 124$) e frazione 010...0 (cioè $b_{21} = 1$, tutti gli altri bit nulli) si ha $1 + 2^{-2} = 1,25$ e $2^{124-127} = 2^{-3}$, quindi

$$\text{valore} = (+1) \times 1,25 \times 2^{-3} = +0,15625.$$

Con la *precisione doppia* (`double`) si usano 1 bit di segno, 11 di esponente e 52 di frazione; la *precisione estesa* (`long double`, 80 bit) ha 1 bit di segno, 15 di esponente, 1 bit di parte intera e 63 di frazione.

6.3 Errori di arrotondamento

L'errore di arrotondamento è intrinseco nel calcolo in virgola mobile.

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 16777217;
5     float b = a;
6     printf("%f\n", b);
7 }
```

```
16777216.000000
```

Il valore $16777217 = 2^{24} + 1$ non è rappresentabile esattamente con un `float`. Allo stesso modo conviene tenere in `double` anche il risultato di un prodotto come `height * width`: salvarlo in un `float` fa perdere precisione.

Osservazione. Il modo più semplice per evitare di accumulare errori è usare numeri in virgola mobile ad alta precisione, cioè `double` al posto di `float`. Sulle CPU moderne non c'è (quasi) penalizzazione di tempo, ma un `double` occupa il doppio della memoria.

7 Tipi enumerati

Le *enumerazioni* sono tipi interi definiti dal programmatore. La definizione inizia con la parola chiave `enum`, eventualmente seguita da un identificatore, e contiene l'elenco dei possibili valori con un nome per ciascuno:

```
1 enum [identificatore] { lista-enumeratori };
```

Esempio.

```
1 enum color { black, red, green, yellow, blue, white = 7, gray };
```

Le costanti elencate valgono 0, 1, 2, 3, 4, 7, 8 (se non si assegna un valore, si continua dal precedente più uno).

```
1 enum color fgColor = blue,           // due variabili
2     bgColor = yellow;               // di tipo enum color
3
4 void setFgColor(enum color fgc);    // funzione con parametro enum color
```

Con le variabili di tipo enumerato si può fare aritmetica ordinaria (`red + red` vale 2). Costanti diverse della stessa enumerazione possono avere lo stesso valore:

```
1 enum signals { OFF, ON, STOP = 0, GO = 1, CLOSED = 0, OPEN = 1 };
```

Esempio.

```
1 #include <stdio.h>
2
3 enum week { sunday, monday, tuesday, wednesday, thursday, friday, saturday };
4
5 int main() {
6     enum week today;
7     today = wednesday;
8     printf("Day %d", today + 1);
9     return 0;
10 }
```

Osservazione. Conviene usare sempre un'enumerazione quando una variabile (in particolare un parametro di funzione) può assumere solo uno tra pochi valori possibili: si evitano errori dovuti a costanti non valide, si documentano i valori ammessi ed è più mnemonico degli interi.

8 Il tipo void

Il tipo `void` indica che non è disponibile alcun valore. Non si possono dichiarare variabili o costanti di questo tipo, ma si usa:

- nelle dichiarazioni di funzioni;
- nelle espressioni `void`;
- nei puntatori a `void` (si vedranno con i puntatori).

Una funzione che non restituisce nulla ha tipo `void`: `void error(int a) {}`. `void` nella lista dei parametri di un prototipo indica che la funzione non ha parametri: `void printMenu(void) {}`. Il compilatore segnala un errore se si chiama `printMenu(3)`.